

CMP-7023B Data Mining - Advanced Data Analysis

100318577

Primary Research Aim

This investigation seeks to comprehensively analyse the `Insurance_Data.csv` dataset to advance understanding of customer profiling and behavioural prediction methodologies within the insurance industry. The research employs a dual-pronged analytical approach, integrating supervised and unsupervised machine learning techniques to classify customers into predefined demographic subtypes and identify natural clustering patterns that emerge from comprehensive customer data analysis.

Dataset Characteristics and Scope

The dataset comprises **5,521 unique customer observations** across **83 distinct variables**, encompassing a comprehensive range of demographic, socioeconomic, and insurance-related attributes. Key variables include household composition metrics, educational attainment levels, income stratification data, social class indicators, and detailed contribution patterns across multiple insurance product categories, including automotive, life, disability, and property insurance policies. This rich, multidimensional dataset provides an extensive foundation for examining complex customer behaviour patterns and developing sophisticated predictive models to inform strategic business decisions in the insurance sector.

The breadth and depth of available variables enable comprehensive analysis of customer characteristics that extend beyond traditional demographic segmentation. This includes behavioural indicators and financial commitment patterns that reflect genuine customer engagement with insurance products. This holistic data structure supports both granular individual-level analysis and broader population-level trend identification, facilitating the development of actionable insights for insurance industry stakeholders.

Primary Research Objectives

1. Supervised Classification Analysis

- Objective:** Develop and validate predictive models capable of accurately classifying customers into predefined demographic subtypes such as "Rural & Low-Income," "Young & Low-Income," "Middle-Class Families," "Wealthy & Affluent," and "Seniors & Retired"
- Methodology:** Implementation of multiple machine learning algorithms, including Random Forest, XGBoost, K-Nearest Neighbours, and Decision Tree approaches
- Evaluation Framework:** Comprehensive performance assessment utilising ROC analysis, AUC metrics, accuracy measures, and cross-validation protocols
- Business Impact:** Enable precise customer targeting, improve risk assessment accuracy, and optimise product recommendation systems

2. Unsupervised Segmentation Discovery

- Objective:** Identify natural customer groupings and behavioural clusters that emerge organically from the data, independent of predefined categorical frameworks
- Methodology:** Application of advanced clustering algorithms, including K-means, hierarchical clustering, and density-based spatial clustering techniques
- Validation Approach:** Cluster stability analysis, silhouette scoring, and business interpretation validation
- Strategic Value:** Uncover hidden market segments, identify cross-selling opportunities, and develop innovative product positioning strategies

Final Workflow Summary

1. Data Dictionary and Data Exploration (Section 1)

The initial phase of the analysis involved the development of a detailed data dictionary accompanied by comprehensive data exploration. The data dictionary was systematically constructed to include standardised and descriptive variable names, enhancing clarity and facilitating interpretability. Each variable was thoroughly described, emphasising its relevance to customer segmentation and profiling within the insurance context. The calculation of missing data statistics complemented this process; the analysis revealed that missing data was negligible, with all variables exhibiting complete observations. Descriptive statistics such as means, standard deviations, minimums, maximums, and percentiles (25%, 50%, 75%) were computed for numerical variables, providing foundational insights into data distribution and variability. To further facilitate understanding of the underlying data structure, Principal Component Analysis (PCA) was employed to reduce the dimensionality of the feature space. This technique enabled visualisation of the data in two dimensions, preserving maximum variance, and illuminated the principal patterns driving data variation. Such exploratory analyses set the stage for subsequent preprocessing and modelling efforts.

2. Categorical and Continuous Variables (Section 2)

A critical aspect of data preparation involved identifying and handling categorical and continuous variables. Categorical features, including "Customer Type," "Household Profile," and various social class indicators, were recognised for their significance in demographic and socioeconomic segmentation. These variables provided discrete classifications that complemented the continuous measures, offering nuanced insights into customer behaviour and preferences. Constant variables—such as income levels, household sizes, education levels, and asset ownership indicators—were selected for their capacity to capture socioeconomic status, financial stability, and behavioural tendencies pertinent to insurance consumption. These features were pivotal inputs for unsupervised clustering and classification models. The strategic selection and delineation of categorical and continuous variables ensured that analyses accurately reflected underlying demographic and economic phenomena, underpinning robust segmentation and prediction outcomes.

3. Handling Missing Data (Section 3)

In the data cleaning phase, an exhaustive examination of missing data was conducted, revealing an absence of missing entries across variables, thus simplifying subsequent procedures. Nevertheless, the proactive strategies adopted involved imputing missing values where necessary, particularly for categorical variables, by replacing absent entries with the mode, thereby preserving the inherent discrete structure of these features. For numerical variables, the median was employed as an imputation measure, effectively mitigating the influence of outliers and maintaining the central tendency of the data. These imputation strategies produced a complete and consistent dataset, minimising biases and errors associated with missing data. The result was a reliable dataset that retained essential patterns and variability, facilitating subsequent outlier detection, clustering, and modelling efforts without the confounding effects of incomplete data.

4. Outlier Detection (IQR and DBSCAN) (Section 4)

The detection and removal of outliers constitute a vital step in refining data quality for clustering analyses. The analysis employed two complementary methodologies: interquartile range (IQR)-based outlier detection and density-based spatial clustering of applications with noise (DBSCAN). The IQR method calculates continuous variables' first and third quartiles, defining outliers as observations falling below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$. This approach effectively identified extreme values likely to distort cluster boundaries. Concurrently, the DBSCAN algorithm was utilised for noise detection based on density estimation. The method identified low-density points considered as outliers by systematically optimising the algorithm's parameters—namely, epsilon (eps) and minimum samples (min_samples)—via k-distance graph analysis and grid search. Removal of these outliers resulted in cleaner datasets, which enhanced the clarity and stability of subsequent clustering processes. The combined outlier detection strategies improved the robustness of customer segmentation, as evidenced by increased silhouette coefficients and more meaningful cluster structures aligned with established industry segments.

5. Unsupervised Analysis (Sections 4.3 onwards)

The unsupervised learning phase centred on applying hierarchical clustering and DBSCAN to identify meaningful customer segments. Hierarchical clustering, implemented through an agglomerative approach, consistently revealed a stable two-cluster solution across different preprocessing variations, underscoring the resilience of the fundamental customer differentiation based on socioeconomic and demographic profiles. Visualisation via PCA plots corroborated the presence of distinct, interpretable clusters, which reflected broad socioeconomic stratifications, such as low-income versus high-income groups. DBSCAN, with its density-based framework, offered a contrasting perspective by uncovering more granular segmentation. In particular, cleaned datasets—such as the IQR-cleaned data—DBSCAN produced a five-cluster solution that closely aligned with predefined categories like "Rural & Low-income" and "Wealthy & Affluent." The method's sensitivity to data density allowed for identifying subgroups within broader segments, providing nuanced insights into customer behaviours and preferences. Visualisation of these clusters in reduced-dimensionality revealed clear separation, facilitating interpretation and validation of the segmentation results against industry knowledge.

6. Supervised Learning (Sections to 9)

Building upon the unsupervised insights, supervised learning models—including decision trees, random forests, and gradient boosting algorithms—were employed to predict customer subtypes derived from clustering outcomes. Decision trees and random forests provided interpretable models, with feature importance analyses highlighting variables such as income, education, and asset ownership as key discriminators among customer segments. Gradient boosting models, particularly XGBoost, were tuned via cross-validation to optimise performance metrics like ROC-AUC, achieving high predictive accuracy and robustness. These models facilitated the translation of unsupervised clusters into actionable customer classifications, enabling targeted marketing, risk stratification, and resource allocation. The supervised frameworks thus served as critical tools for operationalising segmentation insights into real-world business strategies, with model interpretability guiding strategic decision-making.

7. ROC Analysis (Section 10)

The evaluation of model performance via Receiver Operating Characteristic (ROC) analysis constituted a crucial component of the validation process. ROC curves were plotted for each predictive model, and the corresponding Area Under the Curve (AUC) scores were computed to quantify discriminative capability. The analysis revealed that gradient boosting models, especially XGBoost, generally outperformed decision trees and random forests, achieving higher ROC-AUC scores, indicating superior classification quality. Moreover, ROC analysis illuminated the influence of preprocessing strategies, such as IQR and DBSCAN cleaning, on model effectiveness, with models trained on cleaner datasets exhibiting enhanced discriminatory power. This comprehensive evaluation provided a rigorous assessment of the reliability and efficacy of the predictive models, thereby underpinning confidence in their application for customer segmentation and targeted intervention strategies.

1. Data Dictionary for Insurance Data

The following code generates a **comprehensive data dictionary** for the `Insurance` dataset, organised with key information into a markdown table with standardised and refined variable names, ensuring they are more meaningful and representative of the data's content. The process involves:

1.1. **Variable Standardisation:** Original variable names from the dataset are updated and refined to be more descriptive and representative of their contents. This standardisation improves clarity and usability. **Missing** data is identified, calculated and formatted as a number and percentage of missing data per variable.

1.2. **Descriptions:** Generate descriptions based on column names and contents.

1.3. **Descriptive Statistics:** Key statistical metrics are calculated for each column, including mean, minimum, maximum, standard deviation, and percentiles (25%, 50%, 75%). **Categorical Variables** (Non-numeric) are processed to extract meaningful insights. The "Mean" is represented by the mode (most frequent value), while "Min" and "Max" are determined by the smallest and largest non-null values. Missing values are replaced with `Na`. **Numerical Variables** metrics such as mean, min, max, and percentiles are calculated, rounded to two decimal places, and formatted to display whole numbers as integers.

1.4. **Example Values:** Representative examples (up to five unique values) are drawn from each column and formatted for clarity. Whole numeric values (e.g., `1.0`, `2.0`) are displayed as integers (`1`, `2`).

1.5. **Data Dictionary Compilation:** All gathered information—including refined variable names, data types, descriptions, example values, and statistical summaries—is compiled into a structured dictionary, which is converted into a markdown table for presentation.

Markdown Table Output:

The final markdown table includes the following headings:

- **Variable Name:** Standardised and refined column names for clarity.
- **Original Name:** The original column names from the dataset.
- **Type:** Data types such as numeric or categorical (e.g., `int64`, `object`).
- **Number of Entries:** Count of non-null values for each variable.
- **Description:** A meaningful explanation of the column's purpose and content.
- **Example Values:** Up to five unique representative values, formatted for readability.
- **Mean, Std Dev, Min, Max, Percentiles (25%, 50%, 75%):** Statistical summaries for numeric columns, with categorical columns displaying mode, minimum, and maximum categories instead of NaN.

[Disclaimer]: The following code for Data Dictionary generation was supported by Microsoft Copilot/Claude.ai, an AI-powered assistant to assist in Python code generation and debugging.

```
In [1]: # PACKAGES ----
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec
from matplotlib.ticker import FuncFormatter
import seaborn as sns
from scipy import stats

# Suppress warnings
import warnings # reduce cluttered output
warnings.filterwarnings("ignore")

from google.colab import drive
drive.mount('/content/drive')
```

Load Insurance Data

```
In [2]: # LOAD DATA
insurance = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/data_mining_2/Insurance_Data.csv")

print(insurance.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5521 entries, 0 to 5520
Data columns (total 83 columns):
 #   Column                                     Non-Null Count  Dtype
---  -
0   Customer_Type                             5521 non-null   object
1   Number_of_Houses                          5521 non-null   int64
2   Avg_Household_Size                        5521 non-null   int64
3   Avg_Age                                   5507 non-null   object
4   Household_Profile                         5521 non-null   object
5   Married                                  5521 non-null   int64
6   Living_Together                          5521 non-null   int64
7   Other_Relation                            5521 non-null   int64
8   Singles                                  5521 non-null   int64
9   Household_Without_Children                5521 non-null   int64
10  Household_With_Children                   5521 non-null   int64
11  High_Education_Level                      5521 non-null   int64
12  Medium_Education_Level                    5521 non-null   int64
13  Low_Education_Level                       5521 non-null   int64
14  High_Status                              5521 non-null   int64
15  Entrepreneur                              5521 non-null   int64
16  Farmer                                    5521 non-null   int64
17  Middle_Management                        5521 non-null   int64
18  Skilled_Labourers                         5521 non-null   int64
19  Unskilled_Labourers                      5521 non-null   int64
20  Social_Class_A                            5521 non-null   int64
21  Social_Class_B1                           5521 non-null   int64
22  Social_Class_B2                           5521 non-null   int64
23  Social_Class_C                            5521 non-null   int64
24  Social_Class_D                            5521 non-null   int64
25  Rented_House                             5521 non-null   int64
26  Home_Owner                               5521 non-null   int64
27  Owns_One_Car                             5521 non-null   int64
28  Owns_Two_Cars                            5521 non-null   int64
29  Owns_No_Car                              5521 non-null   int64
30  National_Health_Insurance                 5521 non-null   int64
31  Private_Health_Insurance                  5521 non-null   int64
32  Income_Less_Than_30K                      5521 non-null   int64
33  Income_30K_to_45K                         5521 non-null   int64
34  Income_45K_to_75K                         5521 non-null   int64
35  Income_75K_to_122K                       5521 non-null   int64
36  Income_Above_123K                         5521 non-null   int64
37  Average_Income                           5521 non-null   int64
38  Purchasing_Power_Class                    5521 non-null   int64
39  Private_Third_Party_Insurance_Contribution 5512 non-null   object
40  Business_Third_Party_Insurance_Contribution 5521 non-null   object
41  Agricultural_Third_Party_Insurance_Contribution 5521 non-null   object
42  Car_Policy_Contribution                   5521 non-null   int64
43  Delivery_Van_Policy_Contribution           5502 non-null   float64
44  Motorcycle_Scooter_Policy_Contribution     5521 non-null   int64
45  Lorry_Policy_Contribution                  5521 non-null   int64
46  Trailer_Policy_Contribution                5502 non-null   float64
47  Tractor_Policy_Contribution                5521 non-null   int64
48  Agricultural_Machine_Policy_Contribution   5521 non-null   int64
49  Moped_Policy_Contribution                  5469 non-null   float64
50  Life_Insurance_Contribution                5461 non-null   float64
51  Private_Accident_Insurance_Contribution    5469 non-null   float64
52  Family_Accident_Insurance_Contribution     5469 non-null   float64
53  Disability_Insurance_Contribution           5449 non-null   float64
54  Fire_Insurance_Contribution                 5466 non-null   float64
55  Surfboard_Insurance_Contribution            5469 non-null   float64
56  Boat_Insurance_Contribution                 5469 non-null   float64
57  Bicycle_Insurance_Contribution              5469 non-null   float64
58  Property_Insurance_Contribution             5469 non-null   float64
59  Social_Security_Insurance_Contribution      5469 non-null   float64
60  Number_Private_Third_Party_Insurance        5469 non-null   float64
61  Number_Business_Third_Party_Insurance       5469 non-null   float64
62  Number_Agricultural_Third_Party_Insurance   5469 non-null   float64
63  Number_Car_Policies                        5521 non-null   int64
64  Number_Delivery_Van_Policies                5521 non-null   int64
65  Number_Motorcycle_Scooter_Policies          5521 non-null   int64
66  Number_Lorry_Policies                      5521 non-null   int64
67  Number_Trailer_Policies                    5489 non-null   float64
68  Number_Tractor_Policies                    5489 non-null   float64
69  Number_Agricultural_Machine_Policies        5489 non-null   float64
70  Number_Moped_Policies                      5489 non-null   float64
71  Number_Life_Insurances                     5489 non-null   float64
72  Number_Private_Accident_Insurances          5489 non-null   float64
73  Number_Family_Accident_Insurances           5489 non-null   float64
74  Number_Disability_Insurances                5521 non-null   int64
75  Number_Fire_Insurances                     5521 non-null   int64
76  Number_Surfboard_Insurances                 5521 non-null   int64
77  Number_Boat_Insurances                     5521 non-null   int64
78  Number_Bicycle_Insurances                   5521 non-null   int64
79  Number_Property_Insurances                  5472 non-null   float64
80  Number_Social_Security_Insurances           5472 non-null   float64
81  Number_Mobile_Home_Policies                 5447 non-null   float64
82  Mobile_Home_Policies                       5456 non-null   object
dtypes: float64(26), int64(50), object(7)
memory usage: 3.5+ MB
None
```

1.1. Variable Standardisation

```
In [3]: # Create a dictionary to store original variable names
original_names = {}

# Define mapping between original column names and new cleaner names
column_mapping = {
    # Household and customer demographics
    'customer_type': 'customer_type',
    'number_of_houses': 'total_properties_owned',
    'avg_household_size': 'avg_household_size',
    'avg_age': 'avg_age',
    'household_profile': 'household_classification',
    'married': 'married',
    'living_together': 'living_together',
    'other_relation': 'extended_family_members',
    'singles': 'singles',
    'household_without_children': 'childless_households',
    'household_with_children': 'households_with_children',
    # Education Levels
    'high_education_level': 'highly_educated',
    'medium_education_level': 'moderately_educated',
    'low_education_level': 'low_education',
    # Occupation Categories
    'high_status': 'high_status_professionals',
    'entrepreneur': 'entrepreneurs',
    'farmer': 'farmer',
    'middle_management': 'middle_management',
    'skilled_labourers': 'skilled_workers',
    'unskilled_labourers': 'unskilled_workers',
    # Social Class Categories
    'social_class_a': 'social_class_a',
    'social_class_b1': 'social_class_b1',
    'social_class_b2': 'social_class_b2',
    'social_class_c': 'social_class_c',
    'social_class_d': 'social_class_d',
    # Housing and vehicle ownership
    'rented_house': 'rented_house',
    'home_owner': 'home_owner',
    'owns_one_car': 'single_car',
```

```

'owns_two_cars': 'dual_car',
'owns_no_car': 'no_car',
# Health insurance
'national_health_insurance': 'national_health_insurance',
'private_health_insurance': 'private_health_insurance',
# Income categories
'income_less_than_30k': 'income_below_30k',
'income_30k_to_45k': 'income_30k_to_45k',
'income_45k_to_75k': 'income_45k_to_75k',
'income_75k_to_122k': 'income_75k_to_122k',
'income_above_123k': 'income_above_123k',
'average_income': 'mean_income',
'purchasing_power_class': 'purchasing_power_group',
# Financial
'private_third_party_insurance_contribution': 'private_third_party',
'business_third_party_insurance_contribution': 'business_third_party',
'agricultural_third_party_insurance_contribution': 'agricultural_third_party',
'car_policy_contribution': 'car_insurance',
'delivery_van_policy_contribution': 'van_insurance',
'motorcycle_scooter_policy_contribution': 'bike_insurance',
'lorry_policy_contribution': 'lorry_insurance',
'trailer_policy_contribution': 'trailer_insurance',
'tractor_policy_contribution': 'tractor_insurance',
'agricultural_machine_policy_contribution': 'agrimachine_insurance',
'moped_policy_contribution': 'moped_insurance',
'life_insurance_contribution': 'life_insurance',
'private_accident_insurance_contribution': 'private_accident_insurance',
'family_accident_insurance_contribution': 'family_accident_insurance',
'disability_insurance_contribution': 'disability_insurance',
'fire_insurance_contribution': 'fire_insurance',
'surfboard_insurance_contribution': 'surfboard_insurance',
'boat_insurance_contribution': 'boat_insurance',
'bicycle_insurance_contribution': 'bicycle_insurance',
'property_insurance_contribution': 'property_insurance',
'social_security_insurance_contribution': 'social_security_insurance',
# Insurance
'number_private_third_party_insurance': 'private_third_party_policies_count',
'number_business_third_party_insurance': 'business_third_party_policies_count',
'number_agricultural_third_party_insurance': 'agricultural_third_party_policies_count',
'number_car_policies': 'car_policy_count',
'number_delivery_van_policies': 'van_policy_count',
'number_motorcycle_scooter_policies': 'bike_policy_count',
'number_lorry_policies': 'lorry_policy_count',
'number_trailer_policies': 'trailer_policy_count',
'number_tractor_policies': 'tractor_policy_count',
'number_agricultural_machine_policies': 'agrimachine_policy_count',
'number_moped_policies': 'moped_policy_count',
'number_life_insurances': 'life_insurance_count',
'number_private_accident_insurances': 'personal_accident_count',
'number_family_accident_insurances': 'family_accident_count',
'number_disability_insurances': 'disability_insurance_count',
'number_fire_insurances': 'fire_insurance_count',
'number_surfboard_insurances': 'surfboard_insurance_count',
'number_boat_insurances': 'boat_insurance_count',
'number_bicycle_insurances': 'bicycle_insurance_count',
'number_property_insurances': 'property_insurance_count',
'number_social_security_insurances': 'social_security_count',
'number_mobile_home_policies': 'mobile_home_policy_count',
'mobile_home_policies': 'mobile_home_policy'
}

# Store original column names before cleaning
original_column_names = insurance.columns.tolist()

# Clean column names and rename them according to the mapping
insurance.columns = insurance.columns.str.lower().str.replace(' ', '_')

# Store mapping between original and cleaned names
for i, orig_col in enumerate(original_column_names): # [See Disclaimer]
    cleaned_col = insurance.columns[i]
    original_names[cleaned_col] = orig_col

# Apply the column mapping
insurance = insurance.rename(columns=column_mapping)

# Update the original names dictionary to reflect the final mapping
for old_col, new_col in column_mapping.items():
    if old_col in original_names:
        original_names[new_col] = original_names[old_col]
    else:
        original_names[new_col] = old_col

# Create a cleaned version of the dataframe with updated names
ins = insurance.copy()

# Function to check if a float is an integer
def is_integer_value(val):
    if isinstance(val, float):
        return val.is_integer()
    return False

# Function to format values properly
def format_value(val):
    if pd.isna(val):
        return "NA"
    elif is_integer_value(val):
        return int(val)
    elif isinstance(val, float):
        return round(val, 3)
    return val

# Check for missing data
missing_data = ins.isnull().sum()
missing_percentage = (missing_data / len(ins)) * 100

# Create a DataFrame to display missing data information
missing_data_ins = pd.DataFrame({'Missing Values': missing_data, 'Percentage': missing_percentage})
missing_data_ins['Percentage'] = missing_data_ins['Percentage'].apply(lambda x: f"{x:.1f}")
missing_data_ins = missing_data_ins[missing_data_ins['Missing Values'] > 0].sort_values(by='Percentage', ascending=False)

# insurance_data_dictionary.csv was transformed and formatted into the following table [see disclaimer]
ins.to_csv("/content/drive/MyDrive/Colab Notebooks/data_mining_2/insurance_2.csv", index=False)

```

1.2 Generate descriptions based on column names and values

Grouping Insurance-Related Variables for Predictive Analysis

The structured organization of insurance-related variables enhances customer profiling and behavior prediction within the Insurance_Data.csv dataset. Categorizing features—such as household demographics, education levels, occupation types, social class, housing ownership, vehicle ownership, income variables, financial indicators, and insurance policies—streamlines exploratory analysis and machine learning applications, improving interpretability and predictive accuracy. This segmentation supports enhanced customer profiling by identifying socioeconomic traits, aiding classification into predefined subtypes. It also improves feature selection, ensuring that supervised learning models operate with refined and relevant attributes for better predictive performance.

Household Demographics: Captures familial structures and living arrangements, which influence insurance needs:

- Customer classification and segmentation
- Total properties owned by household
- Average household size and primary age bracket
- Lifestyle classification
- Breakdown of household compositions (married, cohabiting, extended family, singles, households with and without children)

Education Levels: Impacts financial literacy and investment behaviors, shaping insurance enrollment trends:

- Proportion of highly educated, moderately educated, and low education individuals

Occupation Categories: Determines income stability and risk exposure, necessitating tailored insurance options:

- High-status professionals
- Entrepreneurs
- Farmers
- Middle management
- Skilled workers
- Unskilled workers

Social Class Categories: Reflects purchasing power and economic stratification, impacting insurance preferences:

- Classification into social classes (A, B1, B2, C, D)

Housing & Vehicle Ownership: Dictates insurance coverage needs based on property assets and transportation:

- Housing status (rented vs. owned)
- Vehicle ownership (single car, dual car, or none)

Health Insurance: Analyzes insurance enrollment trends based on accessibility and private coverage:

- National health insurance participation
- Private health insurance enrollment

Income Variables: Provides insights into purchasing power and policy affordability:

- Income distribution across brackets (below 30K, 30K-45K, 45K-75K, 75K-122K, above 123K)
- Average segment income
- Purchasing power classification

Financial Contributions to Insurance: Consolidates spending patterns and premium contributions:

- Private, business, and agricultural third-party insurance payments
- Vehicle-specific insurance contributions (car, van, bike, lorry, trailer, tractor, agricultural machines, mopeds)
- Life, accident, disability, and fire insurance contributions
- Recreational property and asset insurance (surfboard, boat, bicycle, home, social security policies)

Insurance Policies & Holdings: Tracks consumer choices and coverage trends, offering direct analysis of market demand:

- Total count of held insurance policies for third-party, vehicle, property, life, accident, disability, fire, and recreational insurance
- Mobile home policy tracking

This logical segmentation aligns with the project's aim of analyzing insurance customer behavior by supporting enhanced profiling, improving feature selection, refining clustering strategies, and optimizing actionable insights for policy customization.

```
In [4]: # Generate descriptions based on column names with enhanced details
def generate_description(col_name):
    descriptions = {
        # Household and customer demographics
        "customer_type": "Classification of the customer or customer segmentation category",
        "total_properties_owned": "Total number of properties owned by the household",
        "avg_household_size": "Number of people living in the household",
        "avg_age": "Age bracket of the household's primary members",
        "household_classification": "Lifestyle classification of the household",
        "married": "Number/proportion of married people in the segment (out of 10)",
        "living_together": "Number/proportion of unmarried cohabiting people (out of 10)",
        "extended_family_members": "Number/proportion of people in other family relationships (out of 10)",
        "singles": "Number/proportion of single people (out of 10)",
        "childless_households": "Number/proportion of households without children (out of 10)",
        "households_with_children": "Number/proportion of households with children (out of 10)",

        # Education Levels
        "highly_educated": "Number/proportion with high education (out of 10)",
        "moderately_educated": "Number/proportion with medium education (out of 10)",
        "low_education": "Number/proportion with low education (out of 10)",

        # Occupation Categories
        "high_status_professionals": "Number/proportion in high-status professions (out of 10)",
        "entrepreneurs": "Number/proportion who are entrepreneurs (out of 10)",
        "farmer": "Number/proportion who are farmers (out of 10)",
        "middle_management": "Number/proportion in middle management roles (out of 10)",
        "skilled_workers": "Number/proportion of skilled workers (out of 10)",
        "unskilled_workers": "Number/proportion of unskilled workers (out of 10)",

        # Social Class Categories
        "social_class_a": "Number/proportion in social class A (highest) (out of 10)",
        "social_class_b1": "Number/proportion in social class B1 (out of 10)",
        "social_class_b2": "Number/proportion in social class B2 (out of 10)",
        "social_class_c": "Number/proportion in social class C (out of 10)",
        "social_class_d": "Number/proportion in social class D (lowest) (out of 10)",

        # Housing and vehicle ownership
        "rented_house": "Number/proportion living in rented housing (out of 10)",
        "home_owner": "Number/proportion who own their home (out of 10)",
        "single_car": "Number/proportion owning one car (out of 10)",
        "dual_car": "Number/proportion owning two cars (out of 10)",
        "no_car": "Number/proportion owning no car (out of 10)",

        # Health insurance
        "national_health_insurance": "Number/proportion with national health insurance (out of 10)",
        "private_health_insurance": "Number/proportion with private health insurance (out of 10)",

        # Income Variables
        "income_below_30k": "Number/proportion with income below 30K (out of 10)",
        "income_30k_to_45k": "Number/proportion with income between 30K-45K (out of 10)",
        "income_45k_to_75k": "Number/proportion with income between 45K-75K (out of 10)",
        "income_75k_to_122k": "Number/proportion with income between 75K-122K (out of 10)",
        "income_above_123k": "Number/proportion with income above 123K (out of 10)",
        "mean_income": "Average income value of the segment",
        "purchasing_power_group": "Purchasing power classification category",

        # Financial Contributions to Insurance
        "private_third_party": "Amount contributed to private third-party insurance",
        "business_third_party": "Amount contributed to business third-party insurance",
        "agricultural_third_party": "Amount contributed to agricultural third-party insurance",
        "car_insurance": "Amount contributed to car insurance policies",
        "van_insurance": "Amount contributed to delivery van insurance policies",
        "bike_insurance": "Amount contributed to motorcycle/scooter insurance policies",
        "lorry_insurance": "Amount contributed to lorry insurance policies",
        "trailer_insurance": "Amount contributed to trailer insurance policies",
        "tractor_insurance": "Amount contributed to tractor insurance policies",
        "agrimachine_insurance": "Amount contributed to agricultural machine insurance policies",
        "moped_insurance": "Amount contributed to moped insurance policies",
        "life_insurance": "Amount contributed to life insurance policies",
        "private_accident_insurance": "Amount contributed to personal accident insurance policies",
        "family_accident_insurance": "Amount contributed to family accident insurance policies",
        "disability_insurance": "Amount contributed to disability insurance policies",
        "fire_insurance": "Amount contributed to fire insurance policies",
        "surfboard_insurance": "Amount contributed to surfboard insurance policies",
        "boat_insurance": "Amount contributed to boat insurance policies",
        "bicycle_insurance": "Amount contributed to bicycle insurance policies",
        "property_insurance": "Amount contributed to property insurance policies",
        "social_security_insurance": "Amount contributed to social security insurance policies",
```

```

# Insurance Policy Counts
"private_third_party_policies_count": "Total count of private third-party insurance policies held",
"business_third_party_policies_count": "Total count of business third-party insurance policies held",
"agricultural_third_party_policies_count": "Total count of agricultural third-party insurance policies held",
"car_policy_count": "Total count of car insurance policies held",
"van_policy_count": "Total count of delivery van insurance policies held",
"bike_policy_count": "Total count of motorcycle/scooter insurance policies held",
"lorry_policy_count": "Total count of lorry insurance policies held",
"trailer_policy_count": "Total count of trailer insurance policies held",
"tractor_policy_count": "Total count of tractor insurance policies held",
"agrimachine_policy_count": "Total count of agricultural machine insurance policies held",
"moped_policy_count": "Total count of moped insurance policies held",
"life_insurance_count": "Total count of life insurance policies held",
"personal_accident_count": "Total count of personal accident insurance policies held",
"family_accident_count": "Total count of family accident insurance policies held",
"disability_insurance_count": "Total count of disability insurance policies held",
"fire_insurance_count": "Total count of fire insurance policies held",
"surfboard_insurance_count": "Total count of surfboard insurance policies held",
"boat_insurance_count": "Total count of boat insurance policies held",
"bicycle_insurance_count": "Total count of bicycle insurance policies held",
"property_insurance_count": "Total count of property insurance policies held",
"social_security_count": "Total count of social security insurance policies held",
"mobile_home_policy_count": "Total count of mobile home insurance policies held",
"mobile_home_policy": "Do or Don't have a mobile home insurance policy"
}
return descriptions.get(col_name, f"Represents {col_name.replace('_', ' ')}")

```

1.3 Generate descriptive statistics

```

In [5]: # Generate descriptive statistics
descriptive_stats = insurance.describe(include='all').transpose()

```

1.4 Extracting example values from columns and Data Dictionary Compilation

```

In [6]: # Create data dictionary
data_dictionary = [] # [See Disclaimer]
for col in ins.columns:
    data_type = str(ins.dtypes[col])
    description = generate_description(col)
    # Example values (up to 5 unique examples formatted appropriately)
    example_values = []
    # Access unique values only if it's a Series
    if isinstance(ins[col], pd.Series):
        unique_vals = ins[col].dropna().unique()[:5]
        for val in unique_vals:
            example_values.append(str(format_value(val)))
    else:
        example_values = ["Error getting examples (not a Series)"]

# Calculate statistics for each column
count = len(ins)

# Handle different data types appropriately
if data_type in ['object', 'category']:
    mean = ins[col].mode()[0] if not ins[col].mode().empty else "Na"
    min_val = ins[col].dropna().min() if not ins[col].dropna().empty else "Na"
    max_val = ins[col].dropna().max() if not ins[col].dropna().empty else "Na"
    std_dev = "Na"
    # For categorical data, percentiles don't make sense
    percentile_25 = "Na"
    percentile_50 = "Na"
    percentile_75 = "Na"
else:
    # For numeric columns, calculate statistics
    stats = ins[col].describe()
    mean = format_value(stats.get('mean', "Na"))
    min_val = format_value(stats.get('min', "Na"))
    max_val = format_value(stats.get('max', "Na"))
    std_dev = format_value(stats.get('std', "Na"))
    percentile_25 = format_value(stats.get('25%', "Na"))
    percentile_50 = format_value(stats.get('50%', "Na"))
    percentile_75 = format_value(stats.get('75%', "Na"))

missing_values = ins[col].isna().sum()
missing_percent = round((missing_values / len(ins)) * 100, 2) if len(ins) > 0 else 0
data_dictionary.append({
    "Variable": col,
    "Original Variable": original_names.get(col, "Unknown"),
    "Data Type": data_type,
    "Number of Entries": count,
    "Missing Values": f"{missing_values} ({missing_percent}%)",
    "Description": description,
    "Example Values": ", ".join(example_values),
    "Mean": mean,
    "Min": min_val,
    "Max": max_val,
    "Std Dev": std_dev,
    "25%": percentile_25,
    "50%": percentile_50,
    "75%": percentile_75
})

# Convert to a DataFrame for tabular representation
data_dictionary_df = pd.DataFrame(data_dictionary)
# Save the DataFrame as a CSV file
data_dictionary_df.to_csv("insurance_data_dictionary.csv", index=False)

```